

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – SCENARIO BASED ASSESSMENT

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 2 – Scenario Based Assessment
Weightage:	20% of CIA	Total Marks:	100
CO2 Statement:	Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)		
Student Name:	MANDLA VISWATEJA	Reg. No.:	192311399
Section / Batch:	CSE / 2023	Date:	21/07/2026

Instructions to Students

- Answer BOTH scenario-based questions. Each question carries 50 Marks. Total: 100 Marks.
- Carefully analyze the given scenario and identify the computational problem involved.
- Apply appropriate Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems wherever applicable.
- Clearly justify the chosen solution approach with proper reasoning and analytical interpretation.
- Demonstrate decision-making skills by comparing possible solutions and selecting the most suitable approach.
- Use diagrams, stepwise illustrations, or algorithmic representations wherever necessary.
- Maintain clarity, logical organization, and proper technical presentation throughout the answers.

Question Type	Focus Area	Questions	Marks
Scenario Based Assessment	Analyze real-world scenarios and apply Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems	Q1 - Q2	100 (2 × 50)
GRAND TOTAL:		100 Marks	

SECTION A – SCENARIO BASED ASSESSMENT

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification

Marks Evaluation:

Question No.	Problem Understanding & Context Analysis (25%)	Application of Concepts (30%)	Analytical & Decision-Making Skills (20%)	Solution Design & Approach (15%)	Clarity, Justification & Presentation (10%)	Total Marks (Each – 50)
Q1						
Q2						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Algorithm Analysis Report

Sorting, Searching, and Computational Geometry Algorithms — Case Studies

Question 1 (20 Marks)

A university maintains student records where marks are almost sorted, except for a few recently updated entries.

a) Analyze Selection Sort, Bubble Sort, and Insertion Sort

Selection Sort

- Finds the minimum element and places it in the correct position.
- Number of comparisons is always fixed.
- Performs minimum swaps.
- Time Complexity: $O(n^2)$

Bubble Sort

- Compares adjacent elements and swaps them.
- Stops early if no swaps occur.
- Efficient only if data is nearly sorted.
- Best Case: $O(n)$
- Worst Case: $O(n^2)$

Insertion Sort

- Inserts each element into its correct position.
- Very efficient for nearly sorted data.
- Performs fewer comparisons and shifts.

b) Recommendation

Insertion Sort is the best choice because:

- Dataset is almost sorted.
- Requires fewer comparisons.
- Requires fewer shifts.
- Best-case complexity is $O(n)$.

c) Comparison

Algorithm	Best	Average	Worst	Suitable?
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	No
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Moderate
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Best

Conclusion: Insertion Sort gives minimum computation time for nearly sorted records.

Question 2 (20 Marks)

A logistics company computes distances between every pair of warehouses to identify the closest pair.

a) Explain Brute Force Closest Pair

- Compute the distance between every pair of warehouses.
- Store the minimum distance found.
- Return the closest pair.

Formula:

$$\text{Distance} = \sqrt{[(x_1 - x_2)^2 + (y_1 - y_2)^2]}$$

b) Time Complexity

For n warehouses, number of comparisons:

$$n(n-1) / 2$$

Therefore, Time Complexity = $O(n^2)$

c) Scalability Issue

- Number of comparisons increases rapidly.
- Large memory and CPU usage.
- Slow for thousands of warehouses.

Conclusion: Brute force is simple but inefficient for large datasets.

Question 3 (20 Marks)

A hospital database stores patient records in an unsorted list.

a) Explain Linear Search

- Start from the first record.
- Compare each patient ID with the required ID.
- Stop when found.
- Otherwise continue until the last record.

b) Time Complexity

Case	Complexity
Best	$O(1)$
Average	$O(n)$
Worst	$O(n)$

c) Why Sorting is Not Preferred

- Patient records are frequently updated.
- Sorting after every insertion is expensive.
- Linear Search works directly on unsorted data.
- Suitable when updates are frequent.

Conclusion: *Linear Search is preferred because maintaining sorted order costs more than searching.*

Question 4 (20 Marks)

A GIS application determines the boundary using the Brute Force Convex Hull algorithm.

a) Working

- Select every pair of points.
- Form a line.
- Check whether all other points lie on one side.
- If yes, the edge belongs to the convex hull.
- Repeat for all pairs.

b) Time Complexity

Outer loop = $O(n^2)$ | Checking remaining points = $O(n)$

Overall: $O(n^3)$

c) Inefficiency

- Every point pair is checked.
- Huge number of computations.
- Slow for large datasets.
- Not suitable for GIS applications containing thousands of points.

Conclusion: *Efficient algorithms like Graham Scan or Jarvis March are preferred.*

Question 5 (20 Marks)

A plagiarism detection system uses Brute Force String Matching.

a) Working

- Align the pattern with the beginning of the text.
- Compare characters one by one.
- If mismatch occurs, shift pattern by one position.
- Continue until the end.

- Report matching positions.

b) Time Complexity

Let Text length = n , Pattern length = m

Worst Case: $O(n \times m)$ Best Case: $O(n)$

c) Performance Degradation

- Every character comparison is repeated.
- Large documents require many comparisons.
- Rare pattern occurrence increases execution time.
- Inefficient for large text databases.

Conclusion: Algorithms like KMP, Rabin-Karp, or Boyer-Moore provide much better performance

Pattern:

ALGORITHM

The algorithm compares the pattern with each possible position in the text.

- Position 0 → No match
- Position 1 → No match
- ...
- Position 11 → **Match Found**

Hence, the plagiarism detection system identifies that the phrase "**ALGORITHM**" exists in the document.

5. Time Complexity

Case	Complexity
Best Case	$O(n)$
Average Case	$O(n \times m)$
Worst Case	$O(n \times m)$

Where:

- n = length of text
- m = length of pattern

6. Space Complexity

- $O(1)$ (Constant extra memory)

7. Advantages

- Simple and easy to implement.
- Does not require preprocessing.
- Works well for small documents.
- Suitable for basic plagiarism checking.

8. Limitations

- Slow for large documents.
- Performs many unnecessary comparisons.
- Cannot efficiently detect modified or paraphrased text.
- Less efficient than algorithms like **Knuth–Morris–Pratt (KMP)** or **Boyer–Moore**.

Important Time Complexities to Remember

Algorithm	Best	Average	Worst
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Linear Search	$O(1)$	$O(n)$	$O(n)$
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$
Closest Pair (Brute Force)	$O(n^2)$	$O(n^2)$	$O(n^2)$
Convex Hull (Brute Force)	$O(n^3)$	$O(n^3)$	$O(n^3)$
Brute Force String Matching	$O(n)$	$O(n \times m)$	$O(n \times m)$